

LIBRARY MANAGEMENT SYSTEM - PROJECT REPORT

SUBMITTED BY: ANANYA BAJPAI

REGISTRATION NO.: 26BCY10068

COURSE: INTRODUCTION TO PROBLEM SOLVING AND PROGRAMMING

PLATFORM: VIDYARTHI'S - BUILD YOUR OWN PROJECT

1. Introduction

Many institutional libraries and local reading rooms still manage day-to-day operations with physical paper registers or separate spreadsheets. This project addresses those manual limitations by introducing a lightweight, standalone desktop application. Developed entirely in Python, the software uses Tkinter for a clean graphical user interface and SQLite for handling persistent data storage. The primary goal of this tool is to give a single librarian the ability to record inventory, log student memberships, track book deployments, and automatically compute due dates or late fees out of a combined workspace.

2. Problem Statement

- **The Core Issue:** Managing a library setup through physical records or separate spreadsheets is naturally slow and open to errors. Librarians frequently struggle to quickly check real-time book availability or track down overdue borrowers. Furthermore, small-scale libraries and community reading hubs often lack the budget to purchase expensive, enterprise-grade commercial database systems.
- **Proposed Solution:** I engineered a Python-based local application that centralises book logs, user cards, and active checkouts into an internal relational database. The system automates routine validations—such as calculating fine deadlines or blocking checkouts when physical stock hits zero—thereby eliminating human oversight during basic transactions.

3. Functional Requirements

The application's core functionality is divided into three primary software modules:

- **Inventory & Member Management:** Provides CRUD-style operations to add or remove individual book entries and user records from the system.
- **Issue & Return Engine:** A state-tracking component that updates book availability status and stamps exact return timelines upon user checkouts.
- **Fine Calculation & Storage Layer:** A backend script that evaluates dates to apply financial fines while ensuring all session data is saved to the local SQLite database.

4. Non-Functional Requirements

To ensure optimal software execution, we focused on four performance criteria:

- **Performance:** Because the program queries a local database file directly instead of sending an alert to an external web server, data changes update on-screen almost instantly.
- **Reliability:** The system uses built-in input validation routines. For example, if a user accidentally types text into the "copy count" field, the system catches the irregularity and triggers a warning pop-up rather than allowing a critical application crash.
- **Usability:** The interface layout features a clean, tabbed design segmented into Books, Members, and Issue-Return actions. This lets users execute a complete checkout cycle in just a few clicks.
- **Data Integrity:** To protect past transaction history, the database schema implements foreign key constraints that prevent users from deleting any book or member record currently tied to an active, unreturned loan.

5. System Architecture

The application layout relies on a clear, separated three-tier architectural pattern:

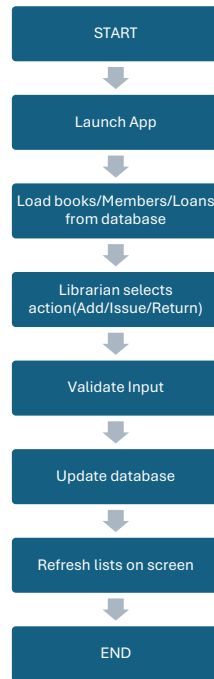
- **Presentation Layer (GUI):** Managed via Tkinter, this layout controls everything the librarian interacts with, including input windows, transaction buttons, and status tables.
- **Logic Layer:** Serves as the functional backbone of the script, evaluating data inputs, computing exact return windows, and calculating fine metrics.
- **Data Layer (Storage):** Driven by an underlying library.db SQLite file. It orchestrates information across three distinct relational tables: books, members, and loans.

6. Design Diagrams

6.1 Use Case Diagram

- **Primary Actor:** Librarian
- **System Actions:**
 1. Catalogue / Remove Book Records
 2. Register / Deregister Members
 3. Issue Books to Active Profiles
 4. Process Book Returns
 5. Monitor Active Borrowers
 6. Calculate Return Windows & Overdue Fees

6.2 Workflow Diagram



6.3 Sequence Diagram (Book Issuance Cycle)

1. The librarian selects a target book and a member profile from the interface, then triggers the "Issue Book" action.
2. The UI layer calls the backend logic to verify that the book's stock count is greater than zero.
3. The engine computes the absolute return deadline using the formula: Current Date + 14 Days.
4. A new transaction row is written directly into the loans database table.
5. The system automatically decrements the book's available_copies value by exactly one.
6. The GUI refreshes its active Listbox views to mirror the new system state and flashes a success dialogue box confirming the return date.

7. Design Decisions & Rationale

- **Language (Python):** Chosen due to its rapid prototyping capabilities and its native, out-of-the-box integration with both UI packages (tkinter) and light database drivers (sqlite3). This eliminates the need for complex external dependencies during evaluation.
- **GUI Choice (Tkinter):** Picked because the project scope specified a standalone desktop app. Tkinter provides an ultra-lightweight footprint that executes natively without requiring a browser engine or local server hosting.
- **Storage Engine (SQLite):** Selected over simple flat files (like CSV or TXT) because we needed relational integrity. SQLite handles primary-foreign key linkages smoothly and guarantees that data won't be corrupted if the application loses power or closes abruptly.
- **UI Design Pattern:** We purposely used simple, list-based selection fields over complicated custom grid tables. This design choice kept our UI rendering code clean, explicit, and easy to maintain while keeping user interactions straightforward.

8. Screenshots / Results

- **Inventory Entry Screen:** UI view showing text field allotments for title registration and initial copy counts.

Library Management System

Books Members Issue / Return

Book Title:
python basics

Number of Copies:
2

Add Book

python programming (4/5 available)

Delete Selected Book

- Loan Validation State: Interface demonstrating multi-item selection to bind a student record to an available book asset.

Library Management System

Books Members Issue / Return

Choose a book:
python programming (4/5 available)
python basics (1/2 available)

Choose a member:
avni

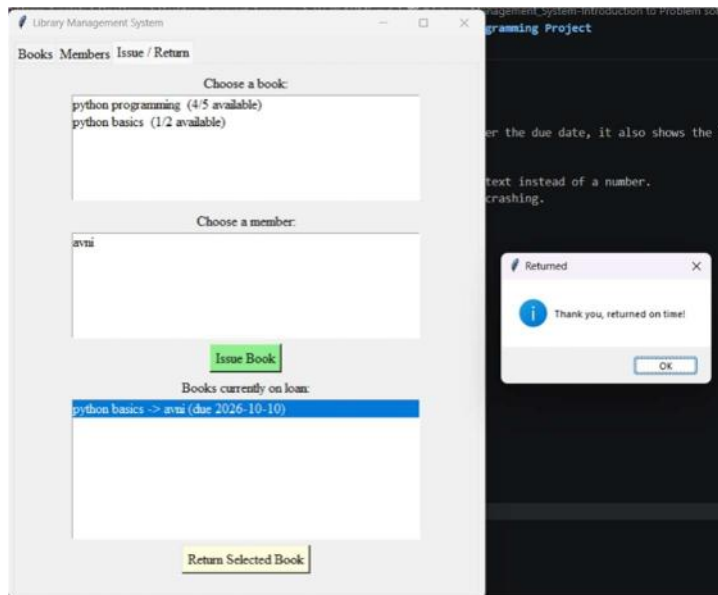
Issue Book

Books currently on loan:
python basics -> avni (due 2026-10-10)

Return Selected Book

Book Issued
Due back on 2026-10-10
OK

- Return Engine & Fine UI: Practical layout capturing how overdue returns trigger an alert box detailing specific financial penalties.



9. Testing Approach

- **Unit Testing:** We directly targeted our database interaction functions (add_book, issue_book, return_book) to confirm that variable modifications commit cleanly to our SQL tables.
- **Boundary Value Testing:** We intentionally pushed faulty arguments—such as registering a book with negative or zero stock copies, or checking out an unavailable asset—to verify that our validation scripts block illegal states.
- **Widget Interaction Testing:** We verified that a user can simultaneously highlight a book row and a member row within the checkout window. To ensure this worked correctly, we set the exportselection=False attribute across our Tkinter Listbox parameters.

10. Challenges Faced

- **Data Mapping in Listboxes:** Mapping a user-clicked row index in a standard Tkinter Listbox back to its specific unique ID in our SQLite database proved difficult without upgrading to a heavy tabular widget.
- **Widget Selection Conflicts:** During initial testing, selecting an item in one listbox automatically cleared selections in adjacent listboxes. Researching Tkinter's internal state management revealed that instances share a selection focus by default. We solved this bug by explicitly declaring `exportselection=False` on our UI widgets.
- **Fine Calculation Logic:** Structuring the date delta formula to dynamically track weekend offsets and output clean penalty values without overcomplicating our return backend required multiple design iterations.

11. Learnings & Key Takeaways

- Developed hands-on experience designing and querying normalised relational database schemas using the SQLite environment.
- Gained an understanding of how Tkinter manages widget states and focus properties, highlighting the importance of configuration attributes when building multi-list layouts.
- Learned the practical value of building minimal, highly optimised software architectures for initial project prototypes rather than over-engineering basic features.

12. Future Enhancements

- ***Search & Filter Optimisation:*** Planning to embed dynamic query search boxes to filter down massive inventories instantly.
- ***Archive Logging:*** Expanding the database schemas to preserve a persistent log of historic, closed loans instead of purging them upon return.
- ***Borrowing Thresholds:*** Adding a rule check to cap the maximum number of simultaneous checkouts allowed per student account.

13. References

- Python Language Core Documentation: python.org
- Tkinter UI Framework Guide: tkdocs.com
- SQLite Database System Specification: sqlite.org